cuDESeq2: GPU acceleration of the DESeq2 differential-expression pipeline

Max Highsmith [1,*], Gregory Koytiger [1,*]

[1]Synthesize Bio

[*]To whom correspondence should be addressed: maxrhighsmith@gmail.com; greg@synthesize.bio

Intended article type: *Original Paper*. Subject: Gene expression.

Abstract

Motivation: DESeq2 is widely used for bulk RNA-seq differential-expression analysis, but its CPU-bound per-gene dispersion and generalized linear model fits make repeated analyses costly.

Results: We present cuDESeq2, a PyTorch reimplementation that batches gene-wise computation on the GPU while following the reference workflow. Comparing eager mode with R DESeq2 1.52.0 across six real designs (7–300 samples, $P = 2$–5), final dispersion has a median p95 relative error of 8.5e-4, significant-gene sets have Jaccard similarity of at least 0.99963, and shrunken LFCs have Pearson correlation of at least 0.99722. The standard Wald path includes count-outlier replacement and refitting. On one A100, the fastest cuDESeq2 mode is 10.6–171.2× faster than one-worker R when the same five stage medians are summed. A separate real-GTEx scaling study runs 54,922 genes across 2,451 samples and a $P = 6$ tissue design in a 22.936-s direct median, 157.4× faster than a one-worker direct R endpoint and 28.0× faster than a 12-worker endpoint.

Availability and implementation: Open source under the MIT license at `https://github.com/synthesizebio/gpu_deseq`; PyTorch and a CUDA GPU are required for the reported acceleration.

Contact: maxrhighsmith@gmail.com; greg@synthesize.bio

Supplementary information: Appended to this manuscript.

1 Introduction

DESeq2 (Love et al., 2014) is a standard method for bulk RNA-seq differential-expression analysis. A typical analysis includes five steps: normalization, dispersion estimation, fitting a negative-binomial GLM, significance testing, and log fold-change shrinkage estimation (Zhu et al., 2019). Many components of the dispersion, GLM, and shrinkage calculations are iterative and independent across genes. In its current implementation, DESeq2 performs these calculations on the CPU with its parallel mode distributing blocks of genes among workers (Fig. 1).

Historically, wall time is a secondary concern for a single DE analysis because the pipeline is only performed a handful of times per project. However, wall time becomes more important when DE must be repeated thousands of times as part of an evaluation loop or comparison of model performance. The bioinformatics community has recently seen growing interest in virtual-cell systems and foundation models for gene-expression data (Theodoris et al., 2023; Cui et al., 2024; Bunne et al., 2024; Adduri et al., 2025; Koytiger et al., 2025). DE is a natural tool for evaluation of such models, and the need to repeat DE analyses in a loop has become more common. Some recent benchmarks have limited wall time by utilizing rank-based tests on normalized expression as less expensive surrogates (Roohani et al., 2025; Wei et al., 2026). A faster implementation of the DESeq2 model would permit rigorous and comprehensive reference analysis to remain in an evaluation loop.

The established route to a faster DESeq2 thus far has been algorithmic and CPU-bound. glmGamPoi (Ahlmann-Eltze and Huber, 2020) accelerates Gamma–Poisson GLM fitting on large, sparse count matrices. However, replacing DESeq2 with glmGamPoi changes the inferential workflow and forgoes DESeq2's automatic outlier replacement, limiting compara-

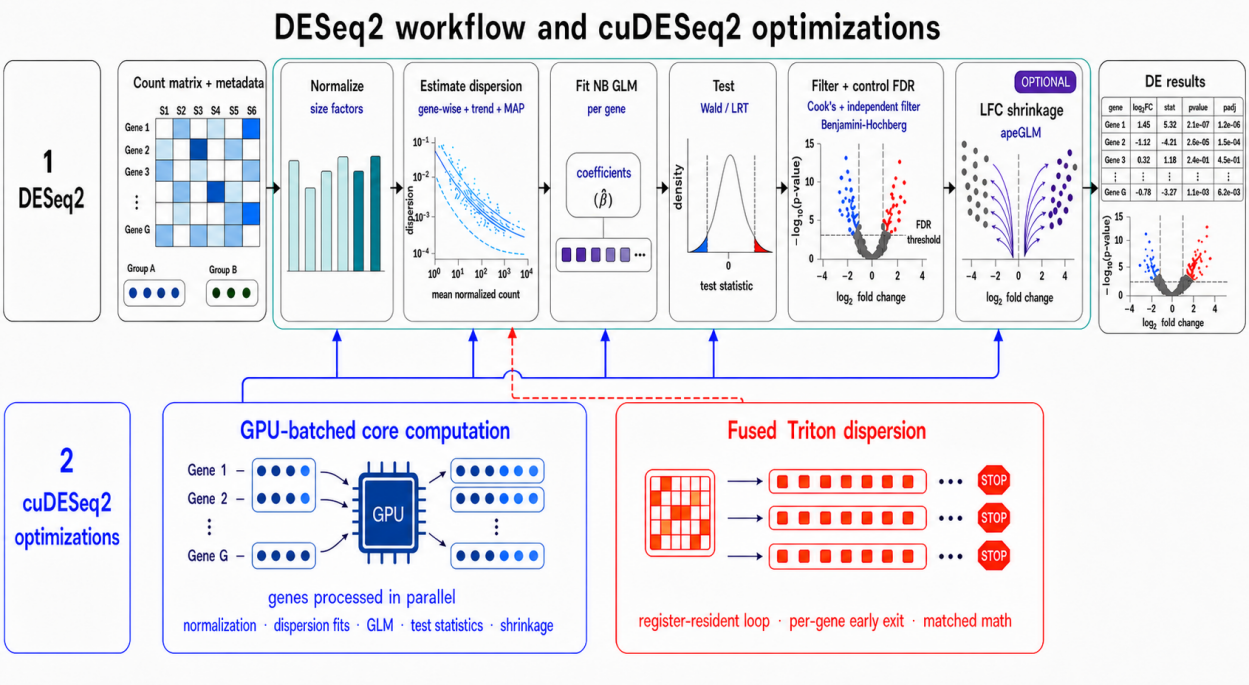

Figure 1: Overview of cuDESeq2. *Top*, the five stages of a DESeq2 run. *Bottom*, where the GPU work goes: DESeq2's per-gene, sequential CPU work (dispersion estimation, the negative-binomial GLM, and apeGLM LFC shrinkage) is batched across all genes (*left*, solid arrows), and the dispersion fit, which is limited mainly by launch latency, has a fully fused Triton implementation (*right*, dashed arrow). A third mode uses CUDA-graph chunked replay; the three modes are compared in Table 1. Every stage is checked against R DESeq2 step by step (Sec. 3.2).

bility with established DESeq2 results and removing a safeguard against influential counts. PyDESeq2 (Muzellec et al., 2023), a Python reimplementation, improves interoperability and speed on large datasets, though by its authors' own account it yields results "similar, but not identical" to DESeq2; GPU acceleration has meanwhile transformed adjacent single-cell workflows, where rapids-singlecell and ScaleSC report 15–20× end-to-end speedups (Hu et al., 2025), but those gains come from the dense and sparse linear-algebra core: PCA, neighbor graphs, and clustering. Our implementation uses PyTorch (Paszke et al., 2019), CUDA graphs (NVIDIA Corporation, 2024), and Triton (Tillet et al., 2019).

In this paper we present the following contributions

(i) Batched GPU implementations of DESeq2's gene-wise dispersion, GLM, and apeGLM shrinkage calculations, including a reference-compatible L-BFGS optimizer that keeps

LFC shrinkage on the GPU.

(ii) A CUDA-graph mode (chunked replay) that removes launch overhead from the dispersion loops, using a capture-safe no-pivot LU in place of cuSOLVER.

(iii) A fully fused Triton kernel: one program per gene runs the whole gradient-ascent loop in registers with per-gene early exit.

(iv) End-to-end and per-step timing benchmarks against R across six real-data designs, simulated sample-count scaling, and real-GTEx full-workflow scaling to 2,451 samples with measured GPU-memory and out-of-memory boundaries.

2 Methods

2.1 Reference workflow and supported scope

DESeq2 (Love et al., 2014) models each gene independently with a negative-binomial GLM. For gene g with counts K_{gi} over samples $i = 1, \ldots, S$, design matrix $X \in \mathbb{R}^{S \times P}$, and size factors s_i, it assumes $K_{gi} \sim \mathrm{NB}(\mu_{gi}, \alpha_g)$ with $\mu_{gi} = s_i \exp(x_i^\top \beta_g)$ and variance $\mu_{gi} + \alpha_g \mu_{gi}^2$. The pipeline is composed of five stages: Normalization, Dispersion Estimation, GLM Fit, Significance Testing, and LFC Shrinkage.

2.1.1 Normalization

Median-of-ratios normalization is implemented with GPU tensor reductions and quantiles. The default `sfType="ratio"` raises, as DESeq2 does, when every gene contains a zero. Users can explicitly select DESeq2's optional `sfType="poscounts"` estimator for such sparse matrices.

2.1.2 Dispersion estimation

Gene-wise dispersion fitting for both the MLE and MAP estimators is batched across genes. The optimization methods used for this stage are described in Sec. 2.2.

2.1.3 GLM fitting and hypothesis testing

Negative-binomial IRLS fits and Wald or likelihood-ratio tests are batched over genes; the implementation details are given in Sec. 2.3.

2.1.4 Significance filtering and result assembly

Cook's filtering, independent filtering, and multiple-testing adjustment are implemented as vectorized array operations. For CUDA analyses, the robust Cook's-distance calculation, thresholding, and count replacement operate on the resident GPU matrices. Independent filtering obtains the rejection counts for its 50 nested thresholds from one p-value ordering and performs the full Benjamini–Hochberg adjustment only at the selected threshold. The standard wrapper performs replacement and refitting when DESeq2's eligibility conditions are met.

2.1.5 LFC shrinkage

The apeGLM optimizer is batched across genes on the GPU; Sec. 2.4 describes the implementation.

2.2 Dispersion acceleration

The dispersion stage is a batched port of DESeq2's `fitDisp` and the supporting `estimateDispersions` logic. Its gene-wise MLE maximizes the Cox–Reid adjusted profile

log-likelihood

$$\ell_{\mathrm{CR}}(\alpha_g) = \sum_{i=1}^{S} \log \mathrm{NB}\big(K_{gi}; \mu_{gi}, \alpha_g\big) \; - \; \tfrac{1}{2} \log \det\big(X^\top W_g X\big), \tag{1}$$

where $W_g = \mathrm{diag}\big(\mu_{gi}/(1 + \alpha_g \mu_{gi})\big)$. We retain DESeq2's initialization, line search, trend fit, MAP shrinkage, and outlier rule. Because dispersion is a significant source of pipeline compute time, we expand optimization beyond basic batching and provide three alternative implementations of the gradient-ascent loop: eager, CUDA-graph, and Triton. The three implementations use the same optimizer decisions and differ only in how those evaluations are scheduled.

Table 1: The three execution modes for the dispersion loop.

Mode	Dispatch	Fidelity	Designs
eager	one kernel per tensor op	reference	any P
graph	replay a captured CUDA graph	bit-identical to eager	any P
Triton	one fused kernel per gene	R parity, not bit-identical	$P \in \{2, \ldots, 6\}$

Both accelerated modes fall back to eager on CPU; Triton also falls back for $P = 1$ or $P > 6$. Triton's agreement with eager is quantified in Sec. 3.2.

CUDA-graph chunked replay. In eager mode each gradient-ascent iteration launches a handful of tiny kernels over 1–10 MB arrays, so the loop is dominated by launch latency. Capturing the loop as a CUDA graph amortizes that cost into a single replay, but two obstacles must be overcome.

The first is early exit. A fixed full-length capture would force every gene to the maximum iteration count, which wastes the typical 15–40-iteration case. To prevent this we capture a K=10-iteration graph that reads and writes a set of persistent state buffers in place, and replay it in a loop with a convergence check between chunks. A chunk always runs to its boundary, so a gene that converges partway through executes further iterations. We selected ten iterations per replay empirically to balance replay overhead against excess iterations after convergence.

The second is cuSOLVER. The Cox–Reid term needs $\log\det(b)$ and $\mathrm{tr}(b^{-1}\,\mathrm{d}b)$ with $b = X^\top W X$, but `slogdet`/`solve` host-synchronize and cannot be captured. Since b is symmetric positive-definite for full-rank X, we replace them with an unrolled no-pivot LU (`_logdet_and_tr_binv_db`) returning both quantities from elementwise and indexing operations alone, matching the `linalg` path to ∼1e-15 for $P = 2 \ldots 6$.

Before capture the chunk is run three times on a side stream, warming the caching allocator and any lazy kernel initialization. Captured graphs are cached on G, S, P, the chunk length K, whether a prior term is enabled, the dtype and the device. Only *whether* there is a prior is selected on the device, because the prior variance lives in the buffer set as a scalar tensor the replay reads, so one graph serves datasets whose prior variances differ. Because the graph uses the same routine, it retains bit-identical parity with the eager mode implementation.

Triton full-loop fusion. Each Triton dispersion fit uses one kernel launch over a grid with one program instance per gene (Fig. 2). Each instance loads its counts, μ, and the design columns once into registers, then runs the iterative gradient-ascent loop register-resident: the Cox–Reid $P \times P$ solve (closed-form 2×2 for P=2, an unrolled no-pivot LU above that), the NB log-likelihood and $\log\alpha$-gradient (`lgamma` and `log1p` from `libdevice`, `digamma` hand-implemented to match ATen's `calc_digamma` through a recurrence to $x \geq 10$ followed by the Bernoulli asymptotic series), and the Armijo line search. A per-gene `while` loop lets each gene exit as soon as it converges, so the kernel does not run every gene to the batch maximum. The kernel is fp64 and covers $P \in \{2, \ldots, 6\}$; `fit_dispersions` gates on the covered range and routes wider designs to the eager path, while the kernel's own launcher raises instead of fitting a truncated model, since it reads a fixed number of design columns. At the gene-wise step, non-converged and high-dispersion genes get the same noIncrease revert and grid fallback as the eager path, computed eagerly after the kernel returns; at the MAP step neither applies, in either mode, because R applies neither. The fused reduction

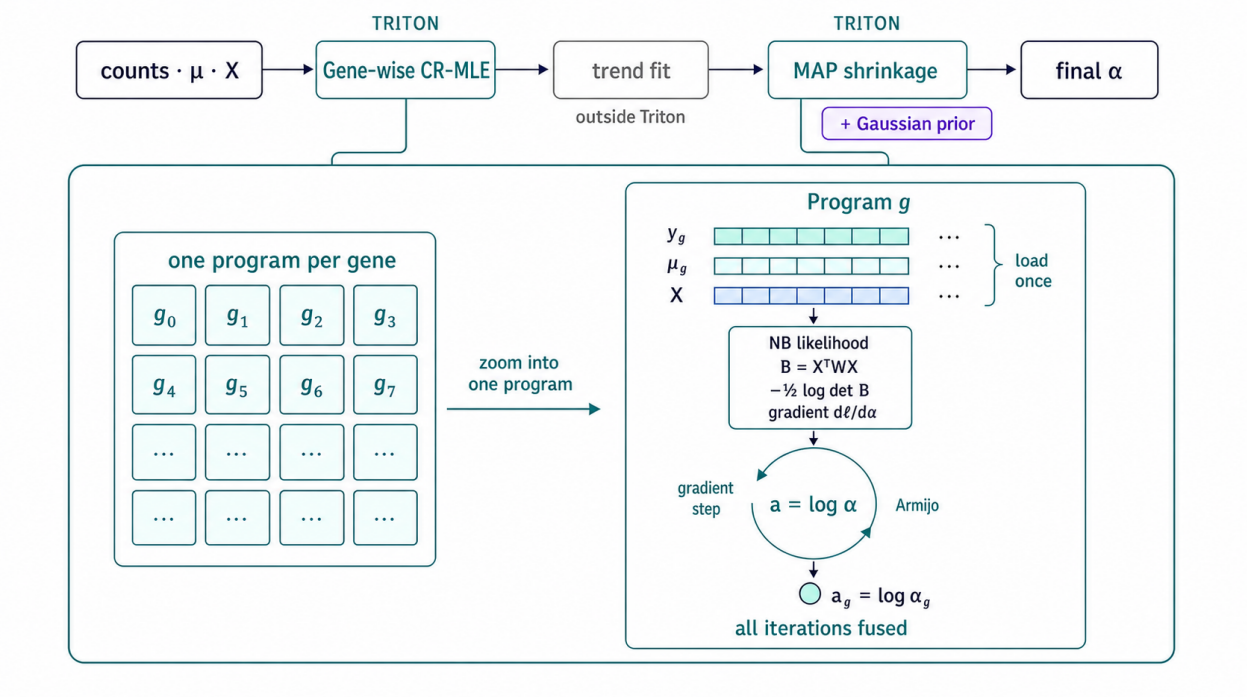

Figure 2: The fused Triton dispersion kernel. The same kernel implementation is invoked separately for the two gene-wise fits of the dispersion pipeline: the Cox–Reid MLE and the MAP shrinkage with a Gaussian prior that follows the trend fit (the trend fit itself runs outside Triton). Each invocation is one kernel launch over a grid with one Triton program instance per gene. Each instance loads $\mathbf{y}_g$, $\boldsymbol{\mu}_g$, and the design $\mathbf{X}$ once, then runs every iteration of the gradient ascent register-resident, with the NB log-likelihood, the Cox–Reid term $-\frac{1}{2}\log\det\mathbf{B}$ ($\mathbf{B} = \mathbf{X}^\top\mathbf{W}\mathbf{X}$), the $\log\alpha$-gradient and the Armijo line search all fused into the loop body. Only the converged $a_g = \log\alpha_g$ and a diagnostic iteration count are written back to global memory. Supplementary Algorithms S1–S2 give the pseudocode.

order and the hand-written `digamma` move the last bits, so this mode is not bit-identical to eager, and Sec. 3.2 reports how far the two diverge on real data.

2.3 GLM acceleration

The GLM is implemented as a batched negative-binomial IRLS solve on the GPU, using one batched factorization across genes at each iteration. Genes that diverge or reach the DESeq2 iteration cap use the same host-side fallback as the reference; only those exceptional gene rows are transferred. Cook's robust dispersion, distance, count replacement, and the affected-gene refit remain on the GPU in CUDA workflows. Wald standard errors retain the $\mu \geq 0.5$ floor in `fitBeta.cpp`; LRTs use the full–reduced deviance difference.

2.4 LFC shrinkage acceleration

apeGLM (Zhu et al., 2019) is a per-gene optimization, so its L-BFGS state, line search, and convergence flags are batched on the GPU. We match the reference optimizer's initialization and stopping rules because extreme, low-count genes can have competing posterior modes. As in `lfcShrink`, only the shrunken coefficient and its standard error are replaced; p-values remain those from the Wald fit. Loss and gradient share the same large gene–sample linear predictor, invariant count/dispersion terms are hoisted out of the optimizer loop, and fitted size factors and Wald inference are reused rather than reconstructed downstream.

2.5 Validation methodology

Reference fixtures are generated by `scripts/generate_r_fixtures.R` against R DESeq2 1.52.0: three single-factor designs at 12×200, 30×500 and 60×2000 (samples $\times$ genes, $P = 2$), a 60×1500 two-factor design $\sim$ batch + condition with a three-level batch ($P = 4$), and a 60×1000 continuous-covariate design ($P = 2$), using the seeds 0, 0, 1, 2, and 3, respectively. Every intermediate is checked against R: size factors, `dispGeneEst`,

`dispFit` (the parametric trend on every fixture, plus the `"local"` and `"mean"` trends on the single-factor ones), `dispMAP`, the final dispersion and its outlier set, β, μ, the hat-matrix diagonals, Cook's distances, Wald SE, the LRT statistic on all five designs, the independent-filtering `padj`, and the apeGLM prior scale and shrunk LFC/SE. Errors are reported as p95 relative values, or absolute values where a quantity crosses zero. Sec. 3.2 reports the measured agreement for the principal quantities and tabulates the five pipeline stages on the real datasets (Table 2). For real-data summaries, max relative error is used for positive size factors; p95 relative error for positive dispersions; p95 absolute error for LFCs, which cross zero; Jaccard similarity of finite padj < 0.05 sets; and Pearson correlation over finite pairs for shrunken LFCs. Empty called-set unions are assigned Jaccard one. Tolerances were fixed at 10^{-6}, 0.10, 0.01, 0.95, and 0.90 in that order. The retained R-parity summary evaluates eager mode, while the retained mode-comparison artifact measures graph and Triton relative to eager. The revised harness records every mode against R separately and requires every mode to pass the same thresholds.

3 Results

3.1 Experimental setup

All GPU measurements are taken on one NVIDIA A100-SXM4-40GB under PyTorch 2.7.1 with CUDA 12.6 and NVIDIA driver 580.126.20. The reference CPU baseline (host A) is R DESeq2 1.52.0 on a 12-vCPU virtual machine, with R 4.6.0 and apeglm 1.34.0. The main real-data suite and repeated real-GTEx GPU sweep use a warm median of five repetitions. GPU stage timers call `torch.cuda.synchronize()` at both boundaries, so no GPU work remains in flight when the clock stops. The matched R–GPU comparison sums the five independently measured stage medians; it is not a directly observed end-to-end duration. A separately collected R experiment times the ordinary `DESeq()` call path directly with one and 12 BiocParallel workers, and is reported without combining it with stage-summed

GPU measurements. The large-GTEx worker comparison likewise uses matched direct calls in separate fresh processes: one observation for each of two cohorts and two worker counts. These four long R endpoints are labeled separately and are not pooled with the five-repetition headline comparison. Separate one-worker staged observations provide the R outputs for GPU parity. OOM-boundary points are also single cold feasibility probes in fresh processes, not timing estimates.

We validate on three real, published RNA-seq datasets sliced into six designs spanning $P = 2$–5 model terms: pasilla (Brooks et al., 2011) (*Drosophila*, n=7, one- and two-factor), airway (Himes et al., 2014) (human, n=8, three design slices), and a 300-sample GTEx (GTEx Consortium, 2020) whole-blood vs. skeletal-muscle contrast (54,922 genes). The first two are distributed as Bioconductor experiment packages; the GTEx counts are recount2's uniform reprocessing of SRA study SRP012682 (Collado-Torres et al., 2017), from which we draw 150 samples per tissue using R seed 1 and convert base coverage to read counts by dividing by the average read length (100 bases for missing or non-positive lengths). The source URLs, byte lengths, SHA-256 values, selected GTEx column indices and sample identifiers, and SHA-256 values of all prepared files are versioned in `validation/data_sources.json` and `validation/prepared_data_manifest.json`. The synthetic scaling study uses fixed-seed negative-binomial simulations where the gene and sample axes can be swept independently. A separate real-GTEx study uses all 9,662 source samples across 54 detailed tissue levels while holding the gene axis fixed at 54,922; nested cohort membership and matrix hashes are stored in `bench/results/gtex_scaling_a100.json`.

3.2 Numerical parity with R

We compared each stage of the pipeline with R DESeq2 1.52.0 across six real-data designs derived from three datasets (Table 2). Agreement was evaluated separately for normalization, dispersion estimation, GLM fitting, significance calls, and LFC shrinkage.

Every eager-mode pipeline stage met its predefined tolerance across all six real-data

Table 2: Agreement with R DESeq2 1.52.0 across six real-data designs. Each design was evaluated independently using a metric appropriate to the indicated pipeline stage.

Substep	Metric	Median	Worst
normalization	max rel. Δ	4.1e-15	5.7e-15
dispersion	p95 rel. Δ	8.5e-4	4.4e-3
glm_fit	p95 $\lvert\Delta\rvert$ (LFC)	2.4e-5	8.1e-5
significance	Jaccard@0.05	1.00000	0.99963
lfc_shrink	Pearson r	0.99999	0.99722

designs. Normalization was effectively identical to R, and the largest dispersion discrepancy was 0.44% at the 95th percentile. Raw LFC estimates differed by at most 8.1×10^{-5} at the 95th percentile, and significant-gene sets had Jaccard similarity of at least 0.99963. Shrunken LFCs had Pearson correlation of at least 0.99722 with R.

3.2.1 Parity in the standard DE plots

In addition to summary statistics we validate against standard diagnostic plots used in DE analysis. Figure 3 displays dispersion-estimation, MA, and volcano plots from both pipelines. The gene-wise MLE cloud, the fitted trend, the shrunken MAP band, and the outlier pattern visually overlap; both implementations call 3,993 genes at padj < 0.05 in this design.

The two pipelines also produce comparable ranked genes (Fig. 4). At the prespecified α=0.05 threshold, four of the six called sets agree exactly and the other two reach 0.99963 and 0.99997.

3.3 Real-data timing versus R

We benchmark all six designs with three GPU execution modes: eager, graph, and Triton. The graph and Triton modes differ from eager cuDESeq2 only in the dispersion stage. We keep the two timing protocols separate: direct R worker scaling in Table 3, and matched stage-summed R–GPU measurements in Table 4.

In the matched stage-summed measurements, Triton is the fastest total mode on all six

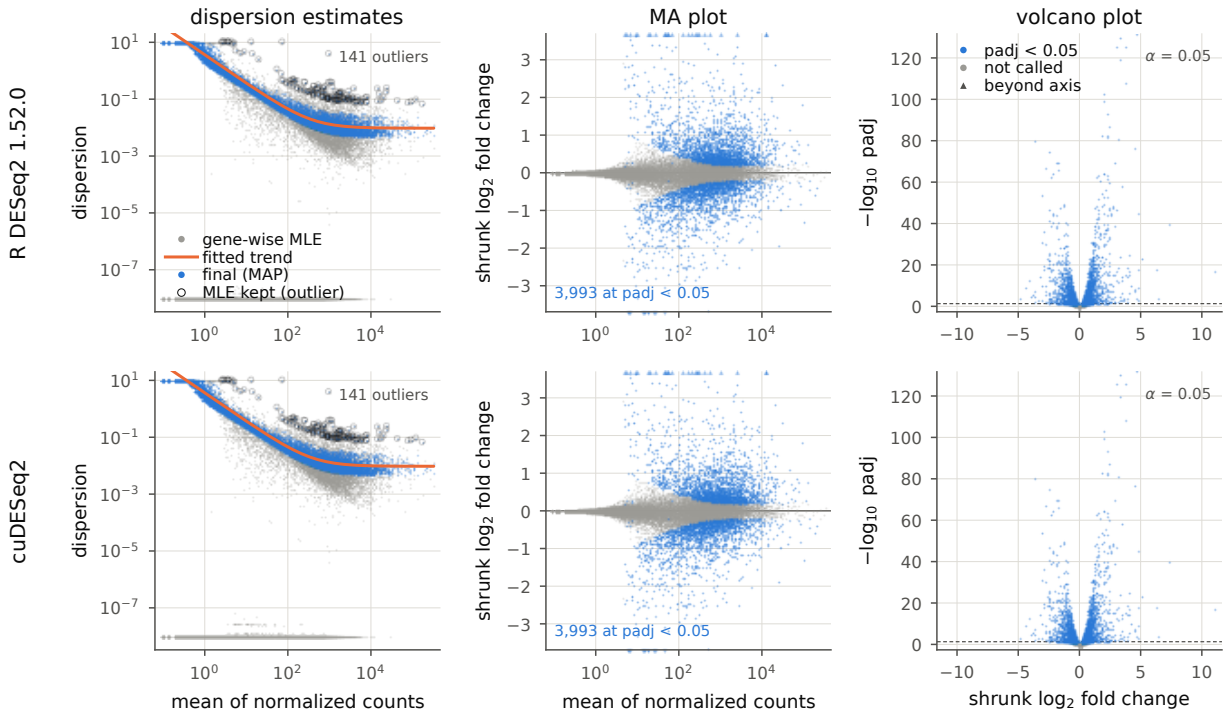

Figure 3: The three standard DESeq2 diagnostics on `airway` ($\sim$ cell + dex, n=8, P=5), drawn from R DESeq2 1.52.0 (top) and cuDESeq2 (bottom) by the same plotting code. *Left*, the `plotDispEsts` view: gene-wise MLE (grey), fitted trend (orange), final estimate (blue), and circled genes whose MLE exceeds the trend by more than two residual SDs and is therefore kept unshrunk. *Center*, MA plot of the apeGLM-shrunk LFC, with out-of-range genes on the boundary as in `plotMA`. *Right*, volcano plot. Axis limits are computed from R's output and reused for cuDESeq2. Reproduce: `validation/export_r_dispersion_details.R` then `validation/make_paper_figures.py`.

Table 3: Direct end-to-end R wall time on the real datasets with one or 12 BiocParallel workers. Each cell is the median of five direct observations of `DESeq()+results()+lfcShrink()`; the `gtex` row includes count replacement and affected-gene refitting. Reproduce: `make container-r-parallel` $\rightarrow$ `bench/results/r_parallel_a100_12worker.json`.

Dataset	P	n	1 worker (s)	12 workers (s)
pasilla	2	7	9.4	7.3
pasilla_2fac	3	7	10.2	7.4
airway_dex	2	8	26.0	11.9
airway_cell	4	8	29.3	10.4
airway	5	8	32.1	12.1
gtex	2	300	276.0	63.4

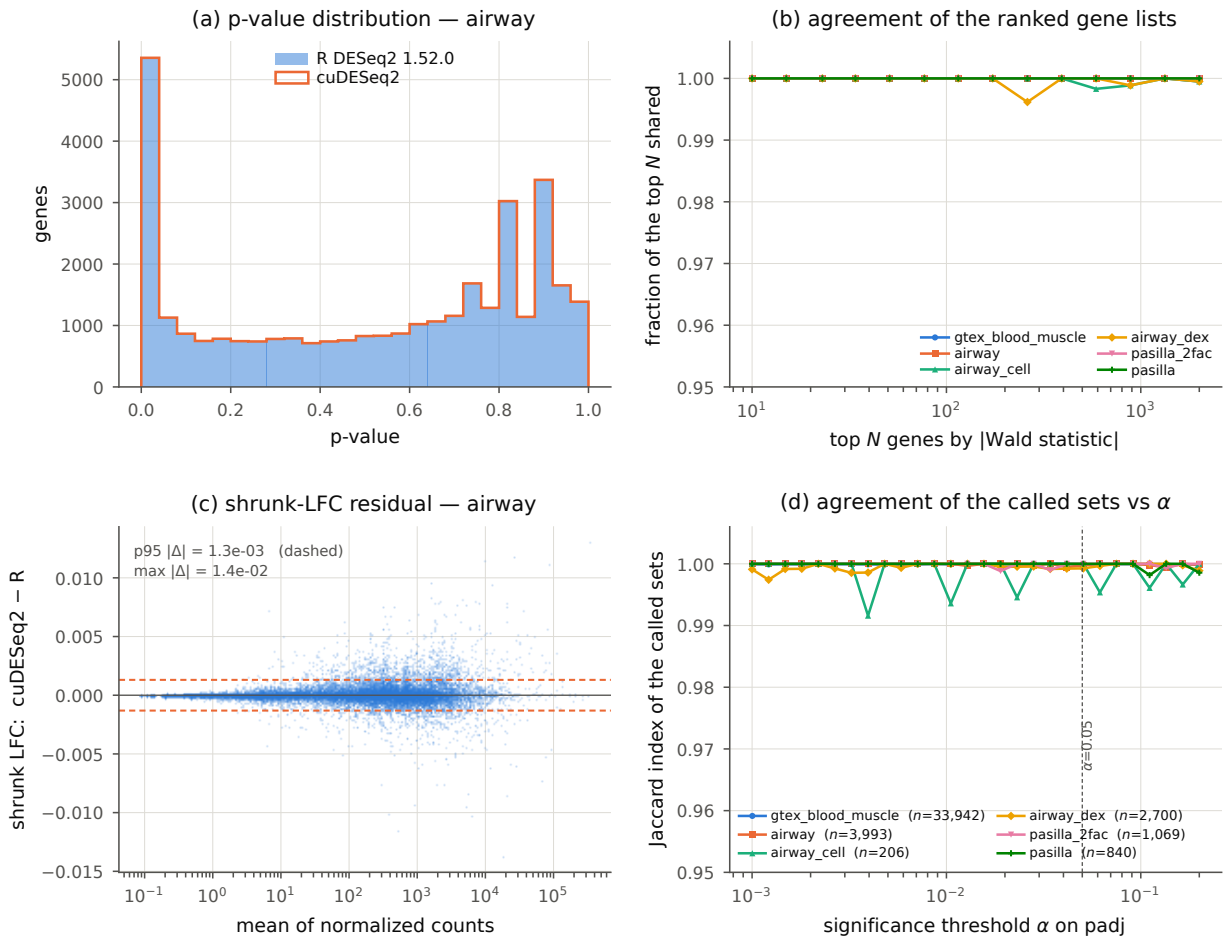

Figure 4: Quantitative agreement across all six designs. *(a)* p-value distributions on `airway`, overlaid. *(b)* Fraction of the top N genes shared by the two rankings, ranked by |Wald statistic| to avoid comparing tie-breaks among genes whose p-value has underflowed in R. *(c)* apeGLM-shrunk LFC residual against expression, showing no mean-dependent bias; dashed lines mark p95 $|\Delta|$. *(d)* Jaccard index of the called sets as the significance threshold moves, with the number of genes R calls at α=0.05 in the legend. The index is granular to $\sim 1/n$, so the small called sets move in visible steps. The `gtex` curve now includes count replacement and refitting in both pipelines.

designs. Against one-worker R using the same stage boundaries, the best GPU mode is 10.6–171.2× faster, with `airway_cell` and `gtex` as the endpoints. Triton has the fastest dispersion substep on all six: fusing the wider Cox–Reid solve (Sec. 2.2) cuts the dispersion substep by 8.9× on `pasilla_2fac` (P=3) and 2.8× on `airway` (P=5) against the eager loop (777→87 ms and 1660→585 ms, same run). `airway`'s dispersion substep also includes the CPU R-stream replay of Sec. 2.2 in every mode. Although it typically lags Triton, graph is bit-identical to eager (Sec. 3.2).

3.4 Sample-count scaling

To measure the sample axis independently of gene count, we fixed 2,000 simulated genes and varied the balanced two-group design from 4 to 2,000 samples (Table 6). The full dispersion stage includes gene-wise estimation, trend fitting, and MAP shrinkage. Iteration counts vary with the simulated input, so wall time need not increase monotonically with sample count. Graph replay is most useful while launch overhead dominates, whereas the fused Triton path remains fastest throughout this sweep.

3.5 Real-GTEx sample, design-width, and memory scaling

The synthetic sweep isolates the sample axis but does not measure the complete workflow on a large real matrix. We therefore materialized all 9,662 samples from the same checksum-pinned recount2 source as a 54,922-gene matrix. Sample order is deterministic, and the first 150 whole-blood and 150 skeletal-muscle samples are byte-identical to the original paper case. The P=2 series appends real samples within those tissues; the design-width series adds the next most abundant tissues with 300 samples per level. Table 7 reports five direct GPU observations after one warm-up and the maximum allocated CUDA memory across the corresponding staged repetitions.

For the nested P=2 series, tripling the real sample count from 300 to 912 increases direct GPU time from 1.753 to 5.430 seconds and peak allocation from 1.98 to 5.99 GiB.

Table 4: Matched per-stage wall time (ms) on the real datasets, one-worker R on the reference host versus the three cuDESeq2 modes. The `glm_fit` timing includes the initial Wald fit and Cook's-distance replacement/refitting, but not the initial dispersion fit; **stage sum** is the sum of the five stage medians and is not a directly observed end-to-end duration. Reproduce: `make container-gpu-benchmark` → `bench/results/timings.json`.

Dataset	Substep	R	eager	graph	Triton
pasilla	dispersion	1782	463	78	21
	glm_fit	2079	42	42	41
	significance	209	40	40	40
	lfc_shrink	5343	420	422	421
	stage sum	**9573**	**966**	**583**	**524**
pasilla_2fac	dispersion	2179	777	174	87
	glm_fit	1495	354	354	352
	significance	184	50	50	49
	lfc_shrink	6082	347	346	347
	stage sum	**10104**	**1528**	**925**	**837**
airway_dex	dispersion	4481	581	141	56
	glm_fit	5171	58	59	57
	significance	1387	82	109	82
	lfc_shrink	14843	573	578	576
	stage sum	**26070**	**1296**	**888**	**773**
airway_cell	dispersion	5456	801	198	62
	glm_fit	3875	39	38	39
	significance	332	78	78	75
	lfc_shrink	20271	2675	2667	2674
	stage sum	**30122**	**3593**	**2982**	**2852**
airway	dispersion	7863	1660	723	585
	glm_fit	4398	62	62	64
	significance	310	74	74	73
	lfc_shrink	20189	673	685	663
	stage sum	**32961**	**2470**	**1545**	**1386**
gtex	dispersion	187567	3574	3612	217
	glm_fit	53421	849	491	432
	significance	456	85	73	95
	lfc_shrink	42474	570	569	929
	stage sum	**287354**	**5081**	**4749**	**1678**

`normalization` omitted (≤4 ms cuDESeq2 on every set); full versioned R records are in `bench/results/timings.json` and `bench/results/r_parallel_a100_12worker.json`. All GPU values come from the same A100 session.

Table 5: Direct end-to-end wall time (s) for the same staged workflow, including dataset construction and host-to-device transfer. Each value is the median of five complete observations; these direct measurements are reported separately from the stage sums in Table 4.

Dataset	R	eager	graph	Triton	best speedup
pasilla	9.945	0.972	0.588	0.525	19.0×
pasilla_2fac	10.462	1.541	0.932	0.848	12.3×
airway_dex	26.678	1.332	0.858	0.790	33.8×
airway_cell	30.564	3.596	3.003	2.870	10.6×
airway	33.063	2.479	1.552	1.401	23.6×
gtex	289.337	5.284	4.910	1.807	160.2×

Table 6: Sample-count scaling of the full dispersion stage (ms) at 2,000 genes. Values are CUDA-synchronized medians of seven measurements after three warm-ups on one A100; the simulator seed is $7919n$. This clean-checkout run used commit `a10620a`, PyTorch 2.7.1, CUDA 12.6, and driver 580.126.20. Reproduce: `benchmarks/bench_cuda_graph.py --sample-sweep` → `benchmarks/results_a100_samplesweep_2026-09-11.json`.

Samples	eager	graph	Triton
4	1075	636	498
6	627	161	32
30	187	68	28
60	238	61	30
200	239	113	34
1000	337	282	23
2000	474	402	90

Table 7: Full-workflow real-GTEx scaling on one A100 at 54,922 genes. Direct wall time is the median of five CUDA-synchronized observations after one warm-up; peak allocation is the maximum over five staged repetitions. P=2–6 uses the fused Triton dispersion path.

Cohort	Samples	P	Direct (s)	Peak GiB
paper-equivalent	300	2	1.753	1.98
2 tissues, balanced	600	2	2.740	3.95
2 tissues, source max.	912	2	5.430	5.99
3 tissues × 300	900	3	6.005	5.92
4 tissues × 300	1,200	4	8.990	8.38
5 tissues × 300	1,500	5	9.898	11.09
6 tissues × 300	1,800	6	14.323	14.77
6 tissues, all samples	2,451	6	22.936	20.10

The widest fused case uses all samples from the six largest tissues: 2,451 samples at $P = 6$ complete in a 22.936-s direct median with 20.10 GiB peak allocation. We also measured the same direct public R pipeline—`DESeqDataSetFromMatrix`, `DESeq`, `results`, and `lfcShrink(type=apeglm)`—with one worker and with `MulticoreParam(12)` in separate fresh processes (Table 8). At 912 samples/P=2, 12 workers reduce the R endpoint from 947.191 to 213.896 seconds, a 4.43× CPU speedup; the GPU remains 174.4× faster than one worker and 39.4× faster than 12. At 2,451 samples/P=6, 12 workers reduce R from 3,609.156 to 642.552 seconds, a 5.62× CPU speedup; the GPU remains 157.4× and 28.0× faster, respectively. These R values are single observations because each complete one-worker call takes 16–60 minutes; they are endpoint comparisons, not repeated headline estimates.

Table 8: Large-GTEx direct endpoint observations. Each R worker setting is one cold observation in a separate process; GPU is the direct median of five warm observations. All columns include dataset construction and the complete workflow. "GPU vs." reports R time divided by GPU time.

Samples	P	GPU (s)	R 1w (s)	R 12w (s)	GPU vs. 1w	GPU vs. 12w
912	2	5.430	947.191	213.896	174.4×	39.4×
2,451	6	22.936	3,609.156	642.552	157.4×	28.0×

Serial and 12-worker R output tables agree to floating-point precision: at P=6 the largest relative dispersion difference is 1.53×10^{-14}, the largest raw- or shrunken-LFC difference is 9.99×10^{-15}, and the significant sets are identical. The independent GPU–R endpoint parity checks use the same five predefined tolerances as Table 2. At P=6, dispersion p95 relative error is 0.00212, raw-LFC p95 absolute error is 3.18×10^{-5}, significant-set Jaccard is 0.99896, and shrunken-LFC Pearson correlation is 0.999994; all five stages pass. The R exporter preserves dispersion immediately after `estimateDispersions`, before Cook's-outlier refitting, so both implementations are compared at the same stage boundary.

Finally, we expanded nested all-sample tissue models beyond the fused kernel's $P \leq 6$ limit. These runs correctly use the eager fallback and are single cold fresh-process feasibility probes (Table 9). P=9 with 3,478 samples passes at 37.09 GiB peak allocation, while P=10

with 3,784 samples fails during dispersion. Thus the observed boundary on this 40-GB A100 lies between those two nested cohorts; it is not a universal sample-count limit because both sample count and design width change.

Table 9: Observed A100 feasibility boundary for nested all-sample GTEx tissue models. Each row is one cold workflow in a fresh process. P> 6 uses eager dispersion fallback; pass-row wall times are not treated as timing estimates.

Tissues	Samples	P	Peak GiB	Status	Failed stage
8	3,147	8	30.98	pass	—
9	3,478	9	37.09	pass	—
10	3,784	10	27.88	OOM	dispersion
12	4,338	12	35.51	OOM	dispersion

4 Discussion

cuDESeq2 shows that the standard DESeq2 analysis pipeline can be moved to a GPU without replacing its statistical model or sacrificing quality. Across six real-data designs, the implementation reproduced the intermediate quantities used by normalization, dispersion estimation, GLM fitting, significance testing, and LFC shrinkage (Table 2), while reducing the matched sum of five stage medians by 10.6–171.2× relative to one-worker R DESeq2 on the A100 benchmark host. These gains make repeated, reference-compatible differential-expression analyses more practical in settings such as model evaluation, where the same analysis is run many times.

The primary speedup comes from replacing sequential per-gene optimization with batched GPU computation. We obtain further speedup through CUDA-graph replay and Triton fusion which reduce the overhead of the dispersion iterations. Speedup varies across designs because time spent in dispersion fitting, shrinkage, and the remaining CPU steps depend on sample size and the design matrix.

We note that our validated scope includes single-factor designs with up to six levels, additive multi-factor designs with up to P=5, and continuous covariates. The fused Triton

path supports $P \in \{2, \ldots, 6\}$ and otherwise uses the eager implementation.

We note that in many instances numerical equivalence depends on subtle implementation details beyond published technical specifications. The dispersion prior, Wald standard errors, and apeGLM shrinkage each exposed behavior that is not fixed by the model specification alone (Sections 2.2, 3.2.1, and 2.4). Reproducing such details is sometimes necessary to obtain DESeq2 equivalent output. For example, we applied DESeq2's minmu floor when forming the GLM weight matrix used to calculate Wald standard errors; without it, genes with near-zero fitted counts receive different standard errors and can produce different p-values.

In other instances we achieve fidelity to the DESeq2 method description with minor differences in floating-point rounding. For example Triton evaluates the same dispersion objective in a fused kernel, but its different floating-point reduction order and digamma implementation introduce small numerical differences from eager PyTorch. The maximum absolute eager–Triton dispersion difference retained by the benchmark is 0.2604 on GTEx and at most 3.8×10^{-7} across the five smaller designs. In such instances, it is worth noting that these differences arise from implementation choices, not from a change to the DESeq2 model, and neither implementation is inherently closer to the intended calculation under that statistical model specification.

This study has several limitations. Measurements use one A100 and one CPU host, so absolute timings and cost effectiveness should not be generalized to other hardware. The primary real-data suite has six designs from three datasets, with five designs containing only seven or eight samples; the separate GTEx scaling study adds larger nested cohorts but not additional source studies. Interactions, observation weights, the `ashr` and normal shrinkage estimators, and the `glmGamPoi` dispersion backend are outside the validated scope. GPU memory use is characterized on one 40-GB A100, not across GPU architectures; the P=8–12 boundary also changes sample count and design width together. Finally, the headline comparison is a matched sum of stage medians, not a direct end-to-end GPU observation;

direct complete-workflow medians are therefore reported separately in Table 5.

In conclusion, cuDESeq2 preserves the DESeq2 analysis while making repeated differential-expression analyses substantially faster on a GPU. Our hope is that this improvement will support the bioinformatics community by enabling reference-compatible DE analysis in workflows that require many repeated comparisons.

Funding

This work was funded by Synthesize Bio.

Conflict of Interest

All authors are affiliated with Synthesize Bio, which developed and maintains the software described here and released it under the MIT license. The authors declare no other competing interests.

Data and software availability

The pasilla benchmark uses the *Drosophila melanogaster* RNA-seq experiment distributed in the Bioconductor `pasilla` 1.40.0 source package (Brooks et al., 2011). The airway benchmark uses the human airway smooth-muscle-cell experiment in the Bioconductor `airway` 1.32.0 source package (Himes et al., 2014). The GTEx whole-blood versus skeletal-muscle benchmark uses recount2 gene-level data for SRA study SRP012682 (`https://recount-opendata.s3.amazonaws.com/recount2/v2/SRP012682/rse_gene.Rdata`) (GTEx Consortium, 2020; Collado-Torres et al., 2017).

Source code, exact source URLs and SHA-256 values, deterministic preparation scripts, prepared-file checksums, benchmark artifacts, and the digest-pinned Docker environment are available at `https://github.com/synthesizebio/gpu_deseq` under the MIT license.

A clean checkout reconstructs all six real-data inputs with `make container-data`; the repository does not redistribute the 1.37-GB GTEx/recount2 object. The reported tables can then be regenerated with the `container-r-reference`, `container-r-parallel`, and `container-gpu-benchmark` Make targets, and checked without untracked caches by `scripts/audit_paper_numbers.py`. The real-GTEx scaling matrix is reconstructed with `bench/extract_gtex_scaling.R`; exact nested source columns, all raw GPU timings, memory observations, OOM records, endpoint R timings, and parity metrics are committed in `bench/results/gtex_scaling_a100.json`.

References

A. K. Adduri, D. Gautam, B. Bevilacqua, et al. Predicting cellular responses to perturbation across diverse contexts with State. *bioRxiv*, 2025. doi: 10.1101/2025.06.26.661135.

Constantin Ahlmann-Eltze and Wolfgang Huber. glmGamPoi: fitting gamma–poisson generalized linear models on single cell count data. *Bioinformatics*, 36(24):5701–5702, 2020. doi: 10.1093/bioinformatics/btaa1009.

Angela N. Brooks, Li Yang, Michael O. Duff, Kasper D. Hansen, Jung W. Park, Sandrine Dudoit, Steven E. Brenner, and Brenton R. Graveley. Conservation of an RNA regulatory map between Drosophila and mammals. *Genome Research*, 21(2):193–202, 2011. doi: 10.1101/gr.108662.110.

Charlotte Bunne, Yusuf Roohani, Yanay Rosen, et al. How to build the virtual cell with artificial intelligence: priorities and opportunities. *Cell*, 187(25):7045–7063, 2024. doi: 10.1016/j.cell.2024.11.015.

Leonardo Collado-Torres, Abhinav Nellore, Kai Kammers, et al. Reproducible RNA-seq analysis using recount2. *Nature Biotechnology*, 35(4):319–321, 2017. doi: 10.1038/nbt.3838.

Haotian Cui, Chloe Wang, Hassaan Maan, Kuan Pang, Fengning Luo, Nan Duan, and Bo Wang. scGPT: toward building a foundation model for single-cell multi-omics using generative AI. *Nature Methods*, 21:1470–1480, 2024. doi: 10.1038/s41592-024-02201-0.

GTEx Consortium. The GTEx consortium atlas of genetic regulatory effects across human tissues. *Science*, 369(6509):1318–1330, 2020. doi: 10.1126/science.aaz1776.

Blanca E. Himes, Xiaofeng Jiang, Peter Wagner, Ruoxi Hu, Qiyu Wang, et al. RNA-Seq transcriptome profiling identifies CRISPLD2 as a glucocorticoid responsive gene that modulates cytokine function in airway smooth muscle cells. *PLoS ONE*, 9(6):e99625, 2014. doi: 10.1371/journal.pone.0099625.

Wenxing Hu, Haotian Zhang, Yu H. Sun, Shaolong Cao, Jake Gagnon, Yuka Moroishi, Yirui Chen, Zhengyu Ouyang, and Baohong Zhang. ScaleSC: a superfast and scalable single-cell RNA-seq data analysis pipeline powered by GPU. *Bioinformatics Advances*, 5(1):vbaf167, 2025. doi: 10.1093/bioadv/vbaf167.

G. Koytiger, A. M. Walsh, V. Marar, et al. Generative genomics accurately predicts future experimental results. *bioRxiv*, 2025. doi: 10.1101/2025.09.08.674753.

Michael I. Love, Wolfgang Huber, and Simon Anders. Moderated estimation of fold change and dispersion for rna-seq data with DESeq2. *Genome Biology*, 15(12):550, 2014. doi: 10.1186/s13059-014-0550-8.

Boris Muzellec, Maria Teleńczuk, Vincent Cabeli, and Mathieu Andreux. PyDESeq2: a python package for bulk RNA-seq differential expression analysis. *Bioinformatics*, 39(9): btad547, 2023. doi: 10.1093/bioinformatics/btad547.

NVIDIA Corporation. CUDA C++ programming guide: CUDA graphs. `https://docs.nvidia.com/cuda/cuda-c-programming-guide/`, 2024. Accessed July 2026.

Adam Paszke et al. PyTorch: An imperative style, high-performance deep learning library. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2019.

Yusuf H. Roohani, Tony J. Hua, Po-Yuan Tung, et al. Virtual Cell Challenge: toward a Turing test for the virtual cell. *Cell*, 188(13):3370–3374, 2025. doi: 10.1016/j.cell.2025.06.008.

Christina V. Theodoris, Ling Xiao, Anant Chopra, et al. Transfer learning enables predictions in network biology. *Nature*, 618:616–624, 2023. doi: 10.1038/s41586-023-06139-9.

Philippe Tillet, H. T. Kung, and David Cox. Triton: an intermediate language and compiler for tiled neural network computations. In *Proc. 3rd ACM SIGPLAN Int. Workshop on Machine Learning and Programming Languages (MAPL)*, pages 10–19, 2019. doi: 10.1145/3315508.3329973.

Zhiting Wei, Yiheng Wang, Yicheng Gao, et al. Benchmarking algorithms for generalizable single-cell perturbation response prediction. *Nature Methods*, 23(2):451–464, 2026. doi: 10.1038/s41592-025-02980-0.

Anqi Zhu, Joseph G. Ibrahim, and Michael I. Love. Heavy-tailed prior distributions for sequence count data: removing the noise and preserving large differences. *Bioinformatics*, 35(12):2084–2092, 2019. doi: 10.1093/bioinformatics/bty895.

Supplementary benchmark details

The benchmark host was a KVM virtual machine with an Intel Xeon CPU at 2.20 GHz, 6 physical cores and 12 logical CPUs, 83 GiB RAM, and Linux 5.10. GPU measurements used one NVIDIA A100-SXM4-40GB. The software environment was PyTorch 2.7.1 with CUDA 12.6 and NVIDIA driver 580.126.20; the R environment was R 4.6.0 with DESeq2 1.52.0 and apeglm 1.34.0. OMP, OpenBLAS, and MKL thread counts were pinned to one for the R measurements. Main-suite and real-GTEx GPU timings are medians of five measured repetitions following an untimed warm-up. The four direct large-GTEx R worker endpoints, two one-worker staged parity references, and four OOM-boundary probes are explicitly single-run measurements. The separate synthetic sample-axis sweep uses seven measured repetitions after three warm-ups and records its software stack in its artifact.

Supplementary algorithms

Supplementary Algorithm S1 gives a high-level view of the fused Triton dispersion fit sketched in Fig. 2; Supplementary Algorithm S2 gives the full kernel, including the Armijo line search, the step-size adaptation, and the Cox–Reid objective/gradient routine shared by the CR–MLE and MAP passes.

Supplementary Algorithm S1 FUSED TRITON DISPERSION FIT (high-level)

Require: counts $\mathbf{Y} \in \mathbb{N}_0^{G \times S}$; fitted means $\boldsymbol{\mu} \in \mathbb{R}_+^{G \times S}$; design $\mathbf{X} \in \mathbb{R}^{S \times P}$; initial dispersions $\boldsymbol{\alpha}^{(0)}$; optional MAP prior.
Ensure: optimized log dispersions $\mathbf{a}$ and iteration counts $\mathbf{c}$.

Launch one Triton program for every gene g in parallel.
    Load $\mathbf{y}_g$, $\boldsymbol{\mu}_g$, and $\mathbf{X}$ once into program-local state.
    Initialize $a \leftarrow \mathrm{clip}(\log \alpha_g^{(0)}, -30, 10)$ and step size $\kappa \leftarrow \kappa_0$.
    repeat inside the same fused program
        Compute the negative-binomial log-likelihood and Cox–Reid adjustment.
        If fitting MAP dispersion, add the Gaussian prior on $a = \log \alpha$.
        Compute the gradient $q \leftarrow \partial \ell / \partial a$.
        Propose the gradient-ascent step $\widetilde{a} \leftarrow a + \kappa q$.
        Accept or reject $\widetilde{a}$ using the Armijo condition; adapt κ.
    until converged or *maxit* is reached.
    Store the final a_g and iteration count c_g once in global memory.

Supplementary Algorithm S2 Detailed fused Triton dispersion kernel (CR–MLE or MAP)

Require: counts $\mathbf{Y} \in \mathbb{N}_0^{G\times S}$; fitted means $\boldsymbol{\mu} \in \mathbb{R}_+^{G\times S}$; design $\mathbf{X} \in \mathbb{R}^{S\times P}$, $P \in \{2,\dots,6\}$; initial dispersions $\boldsymbol{\alpha}^{(0)} \in \mathbb{R}_+^{G}$; compile-time $h \equiv$ HasPrior; prior means $\mathbf{m}$ and variance σ^2 when $h = 1$; initial step bound κ_0.
Ensure: optimized log dispersions $\mathbf{a}$ and iteration counts $\mathbf{c}$.

parallel for Triton program $g \in \{0,\dots,G-1\}$ **do** (one program per gene)
  Load $\mathbf{y}_g$, $\boldsymbol{\mu}_g$, and $\mathbf{X}$ once into program-local state.
  Set $a \leftarrow \mathrm{clip}\Big(\log \alpha_g^{(0)}, -30, 10\Big)$, $\kappa \leftarrow \kappa_0$, $n_{\mathrm{acc}} \leftarrow 0$, and $c \leftarrow 0$.
  Set $(\ell, q) \leftarrow \mathrm{ObjectiveGradient}(a, \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$.
  while $c <$ *maxit* and not converged **do**
    $c \leftarrow c + 1$ and propose $\widetilde{a} \leftarrow a + \kappa q$.
    If $q \neq 0$ and $\widetilde{a} \notin [-30, 10]$, shorten κ so that $\widetilde{a}$ lies on the violated boundary.
    Set $(\widetilde{\ell}, \_) \leftarrow \mathrm{ObjectiveGradient}(\widetilde{a}, \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$.
    Set $r \leftarrow [\,\widetilde{\ell} \geq \ell + \varepsilon\kappa q^2\,]$. (Armijo acceptance flag)
    Select $a' \leftarrow r\,?\,\widetilde{a} : a$, $\ell' \leftarrow r\,?\,\widetilde{\ell} : \ell$, and $\Delta \leftarrow \ell' - \ell$.
    Set $(\_, q') \leftarrow \mathrm{ObjectiveGradient}(a', \mathbf{y}_g, \boldsymbol{\mu}_g, \mathbf{X}, h, m_g, \sigma^2)$ and $q \leftarrow r\,?\,q' : q$.
    Set $n'_{\mathrm{acc}} \leftarrow n_{\mathrm{acc}} + r$ and $\kappa_{\mathrm{acc}} \leftarrow \min(1.1\kappa, \kappa_0)$.
    If r and $n'_{\mathrm{acc}} \bmod 5 = 0$, set $\kappa_{\mathrm{acc}} \leftarrow \kappa_{\mathrm{acc}}/2$.
    Set $\kappa \leftarrow r\,?\,\kappa_{\mathrm{acc}} : \kappa/2$, $n_{\mathrm{acc}} \leftarrow n'_{\mathrm{acc}}$, $a \leftarrow a'$, and $\ell \leftarrow \ell'$.
    Converge if $r \wedge (\Delta < 10^{-6} \ \vee \ a < \log(\mathit{minDisp}/10))$.
  end while
  Store $a_g \leftarrow a$ and $c_g \leftarrow c$ in global memory.
end parallel for
function ObjectiveGradient$(a, \mathbf{y}, \boldsymbol{\mu}, \mathbf{X}, h, m, \sigma^2)$
  Set $\alpha \leftarrow e^a$, $\mathbf{W} \leftarrow \mathrm{diag}(\boldsymbol{\mu}/(1 + \alpha\boldsymbol{\mu}))$, and $\mathbf{B} \leftarrow \mathbf{X}^{\mathsf{T}}\mathbf{W}\mathbf{X}$.
  Compute $\ell \leftarrow \ell_{\mathrm{NB}}(\mathbf{y}; \boldsymbol{\mu}, \alpha) - \frac{1}{2}\log\det(\mathbf{B})$.
  Compute $q \leftarrow \partial\ell/\partial a$, including $\mathrm{tr}\big(\mathbf{B}^{-1}\partial\mathbf{B}/\partial\alpha\big)$ via an unrolled 2×2 formula (P=2) or $P\times P$ LU solve.
  if $h = 1$ **then** $\ell \leftarrow \ell - (a - m)^2/(2\sigma^2)$ and $q \leftarrow q - (a - m)/\sigma^2$. (MAP only)
  return (ℓ, q).
end function